BÀI 3: XÂY DỰNG ỨNG DỤNG SPRING BOOT VÀ THYMELEAF

MỤC TIÊU

- **Hiểu** Spring Bean, tích hợp được Thymeleaf, và cú pháp sử dụng trong themeleaf
- **Áp dụng** được layout page và content page khi sử dụng thymeleaf layout dialect
- **Phát triển** được ứng dụng web đầy đủ các chức năng: quản lý danh sách, thêm, xóa, sửa, tìm kiếm

NỘI DUNG

01

Tổng quan về Spring Bean

02

Làm việc với Controller

03

Sử dụng Thymeleaf

BÀI 3

XÂY DỰNG ỨNG DỤNG SPRING BOOT VÀ THYMELEAF

Tổng quan về Spring Bean

Spring Bean là gì?

- ✓ Spring Bean là các đối tượng java được **Spring IoC Container** quản lý.
- ✓ Được định nghĩa trong **context** của ứng dụng Spring.
- ✓ Là thành phần cốt lõi giúp Spring hỗ trợ Dependency Injection (DI).

Ví dụ: Ứng dụng web quản lý sách, đóng vai trò như **Service Layer (tầng nghiệp vụ)** trong kiến trúc J2EE.

- @Service: Spring sẽ quản lý class này như một bean nghiệp vụ.
- **books**: là một danh sách lưu trữ các đối tượng Book (dùng tạm thay cho database).

```
@Service   nhcuong719
public class BookService {
    private List<Book> books = new ArrayList<>();
    public List<Book> getAllBooks() { return books; }
    public Book getBookById(int id) {...}
    public void addBook(Book book) { books.add(book); }
    public void updateBook(int id, Book updatedBook) {...}
    public void deleteBook(int id) { books.removeIf( Book book -> book.getId() == id); }
}
```

TỔ NG QUAN SPRING BEAN

Spring IoC Container là gì?

IoC (Inversion of Control) Container chịu trách nhiệm:

- Khởi tạo (instantiate)
- Cấu hình (configure)
- Quản lý vòng đời (manage lifecycle) của các bean

Ví dụ:

@Service giúp Spring IoC Container quản lý BookService. Khi cần BookService, chỉ cần @Autowired.

```
@Service
public class BookService {
    private List<Book> books = new ArrayList<>();
    …
}
```

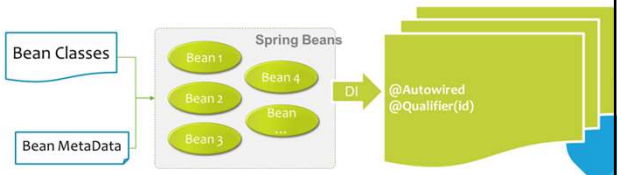
```
@RestController
@RequestMapping("/api/books")
public class BookController {
    @Autowired
    private BookService bookService;
    …
}
```

TỔ NG QUAN SPRING BEAN

Cách tạo Spring Bean

- ✓ Định nghĩa Bean bằng **Annotation** (@Component, @Service, @Controller, @Repository), **Java Config** (**@configuration, @bean**), hoặc **XML Config.**

- @Component
- @Controller
- @Service
- @Repository
- @Configuration
- @Bean



TỔ NG QUAN SPRING BEAN

Sử dụng Spring Bean

- ✓ Sử dụng @Autowired để inject Bean vào class khác.

Ví dụ:

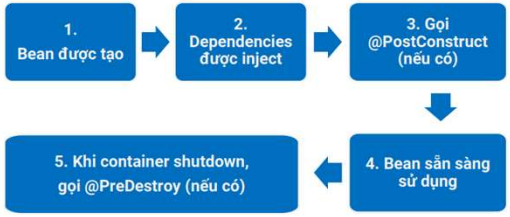
```
@RestController
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping("/user")
    public String getUser() {
        return userService.getUser();
    }
}
```

```
@Service
public class UserService {
    public String getUser() {
        return "Người dùng: Nguyễn Văn A";
    }
}
```

TỔ NG QUAN SPRING BEAN

Vòng đời của Spring Bean

✓ Các giai đoạn chính

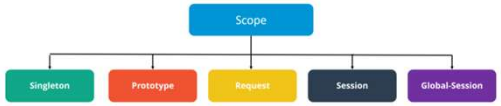


TỔ NG QUAN SPRING BEAN

Phạm vi (scope) của Bean

✓ Phạm vi của bean quy định vòng đời và số lượng instance của một Spring Bean được tạo và sử dụng trong ứng dụng.

✓ Các phạm vi phổ biến:



✓ Cách khai báo phạm vi bean: Dùng annotation @Scope("scope_name") kết hợp với @Component, @Service...

TỔ NG QUAN SPRING BEAN

Ví dụ sử dụng scope

✅ Ví dụ 1: Singleton (Service – NÊN DÙNG)

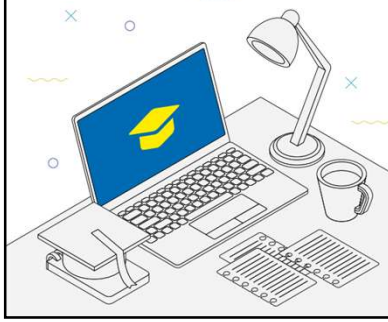
```java
@Service
public class BookService {
    private List<Book> books = new ArrayList<>();
}
```

✅ Ví dụ 2: Prototype (DTO / Model

```java
@Component
@Scope("prototype")
public class Book {
    private String title;
}
```

✅ Ví dụ 3: Request Scope (Request Data)

```java
@Component
@Scope("request")
public class RequestInfo {
    private String requestId;
}
```



BÀI 3

XÂY DỰNG ỨNG DỤNG SPRING BOOT VÀ THYMELEAF

Làm việc với Controller

LÀM VIỆC VỚI CONTROLLER

Định nghĩa

- ✓ Controller là một thành phần trong Spring Boot dùng để xử lý các yêu cầu HTTP.
- ✓ Controller kết nối giữa giao diện người dùng và logic nghiệp vụ.
- ✓ Sử dụng các annotation: @Controller @RestController.
- ✓ Các annotation xử lý HTTP phổ biến:

@RequestMapping | @GetMapping | @PostMapping | @PutMapping | @DeleteMapping

LÀM VIỆC VỚI CONTROLLER

Ví dụ cơ bản:

/api/books

```
@RestController
@RequestMapping("/api")
public class BookController {
    @GetMapping("/books")
    public List<Book> getBooks() {
      return Arrays.asList(new Book(2025,"J2EE","Huy Cuong"));
    }
}
```

Khi truy cập http://localhost:8080/api/books, server sẽ trả về **JSON của sách**.

```
[
  {
    "id":2025,
    "title":"J2EE",
    "author":"Huy Cuong"
  }
]
```

/home

```
@Controller

public class HomeController {
    no usages
    @GetMapping("/home")
    public String Index() {
        return "index";
    }
}
```

- resources
  - static
  - templates
    - index.html
  - application.properties

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<h1>Welcome to the Demo Application</h1>
</body>
</html>
```

LÀM VIỆC VỚI CONTROLLER

Các Annotations phổ biến: @RequestParam

- ✓ **@RequestParam**: Lấy dữ liệu từ query string (tham số trong URL).
- ✓ Ví dụ:
  http://localhost:8080/api/books?**title**=Spring&**author**=Cuong

```
@GetMapping("/books")
public String getBookByQuery(@RequestParam String title,
@RequestParam String author) {
    return "Title: " + title + ", Author: " + author;
}
```

LÀM VIỆC VỚI CONTROLLER

Các Annotations phổ biến: @Requestparam

- ✓ Các thuộc tính:
  - **value**: Tên tham số trong query string.
  - **required**: Xác định tham số có bắt buộc hay không (true mặc định).
  - **defaultValue**: Giá trị mặc định nếu tham số không được cung cấp.

```
@GetMapping("/books")
public String getBookWithDefaults( @RequestParam(value = "title", required =
false, defaultValue = "Unknown") String title,
                                   @RequestParam(value = "author", required =
false, defaultValue = "Anonymous") String author) {
    return "Title: " + title + ", Author: " + author;
}
```

LÀM VIỆC VỚI CONTROLLER

Các Annotations phổ biến: @PathVariable

- ✓ **@PathVariable**: Lấy dữ liệu từ URL path (phần động trong URL). Không hỗ trợ giá trị mặc định.
- ✓ Ví dụ: http://localhost:8080/api/books/Spring/Cuong

```
@GetMapping("/books/{title}/{author}")
public String getBookByPath(@PathVariable String title, @PathVariable String author) {
    return "Title: " + title + ", Author: " + author;
}
```

- ✓ Các thuộc tính:
  - **value**: Tên của biến trong path.

```
@GetMapping("/books/{bookTitle}/{bookAuthor}")
public String getBookWithCustomPath( @PathVariable("bookTitle") String title,

@PathVariable("bookAuthor") String author)
{
    return "Title: " + title + ", Author: " + author;
}
```

LÀM VIỆC VỚI CONTROLLER

Chuyển hướng Redirect

- ✓ Khi muốn gọi một xử lý khác trong ứng dụng mà không làm thay đổi URL. Chuyển hướng yêu cầu HTTP từ một URL này sang URL khác.
- ✓ **Ví dụ:** Ban đầu truy cập /redirect, sau đó trình duyệt chuyển hướng đến /newPage.

```
@GetMapping("/redirect")
public String redirect() {
    return "redirect:/newPage";
}
@GetMapping("/newPage")
public String newPage() {
    return "newPage"; // Render trang newPage.html
}
```

LÀM VIỆC VỚI CONTROLLER

Chuyển tiếp Forward

- ✓ Chuyển tiếp yêu cầu HTTP từ một URL này sang một URL khác trong cùng ứng dụng (trong nội bộ server).
- ✓ Ví dụ: Truy cập ***/forward***, nhưng nội dung hiển thị là của ***/newPage***, và URL trình duyệt vẫn là /forward.

```
@GetMapping("/forward")
public String forward() {
    return "forward:/newPage";
}

@GetMapping("/newPage")
public String newPage() {
    return "newPage"; // Render trang newPage.html
}
```

BÀI 3

XÂY DỰNG ỨNG DỤNG SPRING BOOT VÀ THYMELEAF

Sử dụng Thymeleaf

SỬ DỤNG THYMELEAF

❑ Thymeleaf là gì?

✓ Thymeleaf là một Java Template Engine. Có nhiệm vụ xử lý và tạo ra các file HTML, XML, v.v..

✓ **Đặc điểm:**

- Tạo giao diện tự nhiên: nhờ kết hợp dữ liệu và template + quy tắc để sinh ra một file HTML chứa đầy đủ thông tin.
- Tích hợp mạnh mẽ với Spring Framework.

SỬ DỤNG THYMELEAF

❑ Cấu hình Thymeleaf

✓ Thêm dependency trong pom.xml với Maven

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starterthymeleaf</artifactId>
</dependency>
```

✓ Thư mục chứa template src/main/resources/**templates**

SỬ DỤNG THYMELEAF

❑ Ví dụ Controller trả về Thymeleaf

✓ Truy cập http://localhost:8080/home sẽ hiển thị: Hello Thymeleaf

Hello Thymeleaf

SỬ DỤNG THYMELEAF

❑ Cú pháp đơn giản của Thymeleaf

✓ Sử dụng các thẻ có thuộc tính bắt đầu bằng th: để xử lý dữ liệu động.

<p th:text = "Phát triển ứng dụng J2EE- Nguyễn Huy Cường" >

<th:block>...</th:block>: Dùng để nhóm thẻ HTML mà không ảnh hưởng đến cấu trúc.

✓ **Biến**

${...}: Giá trị của một biến.

*{...}: Giá trị của một biến được chỉ định (form).

#{...}: Giá trị message.

@{...}: Lấy đường dẫn URL dựa theo context của server.

SỬ DỤNG THYMELEAF

❑ Cú pháp đơn giản của Thymeleaf

✓ **Các thuộc tính phổ biến:**

- th:text: hiển thị chuỗi thuần.
- th:utext: hiển thị chuỗi chứa HTML.
- th:each: sử dụng vòng lặp để hiển thị danh sách.
- th:if, th:unless: điều kiện hiển thị.

```
<ul>
    <li th:each="item : ${items}" th:text="${item}"></li>
</ul>
```

SỬ DỤNG THYMELEAF

❑ Truyền dữ liệu từ Controller sang view

✓ Spring Boot sử dụng các đối tượng như **Model**, **ModelMap**, hoặc **ModelAndView** để truyền dữ liệu từ **Controller** đến **View**

- Sử dụng Model ở Controller.

```
@GetMapping("/example")
public String methodName(Model model) {
    model.addAttribute("key", value); // Truyền dữ liệu
    return "viewName"; // Tên file HTML (view)
}
```

- Đọc dữ liệu từ templates/viewName.

```
<h1 th:text="${key}"></h1>
```

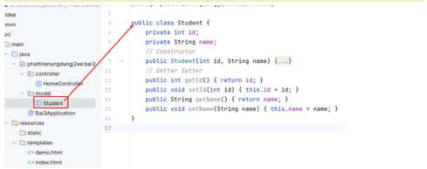
SỬ DỤNG THYMELEAF

❑ Demo – Truyền dữ liệu qua Model

✓ Truyền dữ liệu qua **Model** với phương thức **addAttribute.**

Thực hiện ở controller

```
@GetMapping("/demo")
public String demoPage(Model model) {
    Student student = new Student(1, "Nguyễn Văn A");
model.addAttribute("student", student);
model.addAttribute("message", "Welcome Thymeleaf!");
    return "demo";
}
```

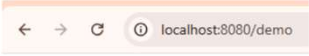


Thực hiện ở View (demo.html)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8" />
    <title>Demo Thymeleaf</title>
</head>
<body>
<!— ${} lấy giá trị biến message —>
<p th:text="${message}">Default message</p>
<!— *{} lấy giá trị biến trong object student —>
<p th:text="*{name}" th:object="${student}">Tên
sinh viên</p>
<!— #{} lấy giá trị từ message.properties —>
<p th:text="#{welcome}">Welcome default</p>
<!— @{} tạo URL —>
<a th:href="@{/home}">Trang chủ</a>
</body>
</html>
```

SỬ DỤNG THYMELEAF

❑ Demo – kết quả truyền dữ liệu qua Model



Welcome Thymeleaf!

Nguyễn Văn A

??welcome_vi_VN??

Trang chủ

SỬ DỤNG THYMELEAF

- Demo – Ứng dụng quản lý sách
  - ✓ Viết ứng dụng sử dụng **thymeleaf** với các giao diện:
    - books.html
    - add-book.html
    - edit-book.html
  - ✓ Hướng dẫn:

Thực hiện giống quản lý sách (bài học trước)	Thêm các giao diện theo yêu cầu (sử dụng thymeleaf) với dự án	Điều chỉnh các action trong controller phù hợp

SỬ DỤNG THYMELEAF

- Demo – Ứng dụng quản lý sách

SỬ DỤNG THYMELEAF

- Demo – Thực hiện Bookservice

```java
@Service
public class BookService {
    private List<Book> books = new ArrayList<>();
    private Long nextId = 1L;
    public List<Book> getAllBooks() {
        return books;
    }

    public void addBook(Book book) {
        book.setId(nextId++);
        books.add(book);
    }

    public Optional<Book> getBookById(Long id) {
        return books.stream().filter(book -> book.getId().equals(id)).findFirst();
    }
    public void updateBook(Book updatedBook) {
        books.stream()
                .filter(book -> book.getId().equals(updatedBook.getId()))
                .findFirst()
                .ifPresent(book -> {
                    book.setTitle(updatedBook.getTitle());
                    book.setAuthor(updatedBook.getAuthor());
                });
    }
    public void deleteBook(Long id) {
        books.removeIf(book -> book.getId().equals(id));
    }
}
```

SỬ DỤNG THYMELEAF

- Demo – Thực hiện Bookcontroller

```java
@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;
    @GetMapping
    public String listBooks(Model model) {
        model.addAttribute("books",
bookService.getAllBooks());
        return "books";
    }
    // Hiển thị form thêm sách
    @GetMapping("/add")
    public String addBookForm(Model model) {
        model.addAttribute("book", new Book());
        return "add-book";
    }
// Thêm sách mới
    @PostMapping("/add")
    public String addBook(@ModelAttribute Book
book) {
        bookService.addBook(book);
        return "redirect:/books";
    }
```

```java
    // Hiển thị form sửa sách
    @GetMapping("/edit/{id}")
    public String editBookForm(@PathVariable Long id,
Model model) {
        bookService.getBookById(id).ifPresent(book ->
model.addAttribute("book", book));
        return "edit-book";
    }
    // Cập nhật sách
    @PostMapping("/edit")
    public String updateBook(@ModelAttribute Book book) {
        bookService.updateBook(book);
        return "redirect:/books";
    }
    // Xóa sách
    @GetMapping("/delete/{id}")
    public String deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return "redirect:/books";
    }
}
```

SỬ DỤNG THYMELEAF

- Demo – Thực hiện template/books.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Danh sách sách</title>
</head>
<body>
<h1>Danh sách sách</h1>
<a href="/books/add">Thêm sách mới</a>
<table border="1">
  <thead>
  <tr>
    <th>ID</th>
    <th>Tiêu đề</th>
    <th>Tác giả</th>
    <th>Hành động</th>
  </tr>
  </thead>
  <tbody>
  <tr th:each="book : ${books}">
    <td th:text="${book.id}"></td>
    <td th:text="${book.title}"></td>
    <td th:text="${book.author}"></td>
    <td>
      <a th:href="@{/books/edit/{id}(id=${book.id})}">Sửa</a>
      <a th:href="@{/books/delete/{id}(id=${book.id})}">Xóa</a>
    </td>
  </tr>
  </tbody>
</table>
</body>
</html>
```

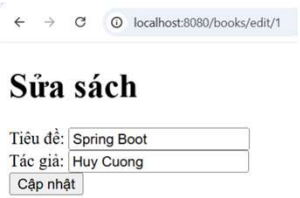
SỬ DỤNG THYMELEAF

- Demo – Thực hiện template/add-book.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Thêm sách mới</title>
</head>
<body>
<h1>Thêm sách mới</h1>
<form action="/books/add" method="post" th:object="${book}">
  <label for="title">Tiêu đề:</label>
  <input type="text" id="title" th:field="*{title}" required />
  <br />
  <label for="author">Tác giả:</label>
  <input type="text" id="author" th:field="*{author}" required />
  <br />
  <button type="submit">Thêm</button>
</form>
</body>
</html>
```

SỬ DỤNG THYMELEAF

- Demo – Thực hiện template/edit-book.html




```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Sửa sách</title>
</head>
<body>
<h1>Sửa sách</h1>
<form action="/books/edit" method="post" th:object="${book}">
  <input type="hidden" th:field="*{id}" />
  <label for="title">Tiêu đề:</label>
  <input type="text" id="title" th:field="*{title}" required />
  <br />
  <label for="author">Tác giả:</label>
  <input type="text" id="author" th:field="*{author}" required />
  <br />
  <button type="submit">Cập nhật</button>
</form>
</body>
</html>
```

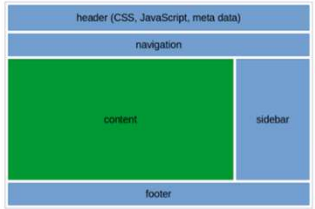
SỬ DỤNG THYMELEAF

- Layout page và Content page

✓ Sử dụng layout page (trang bố cục) và content page (trang nội dung) để tổ chức giao diện một cách gọn gàng và dễ quản lý, tương tự như việc sử dụng các template trong các hệ thống khác.

✓ **Layout Page (Trang bố cục):**

- Định nghĩa cấu trúc chung của trang web (header, footer, sidebar, v.v.).
- Các vùng nội dung (content area) được định nghĩa sẵn để các trang con (content page) nhúng nội dung.



✓ **Content Page (Trang nội dung):**

- Là trang nội dung cụ thể, sẽ được chèn vào layout page.
- Giúp giao diện web gọn gàng, dễ quản lý.

SỬ DỤNG THYMELEAF

- Cài đặt Thymeleaf Layout Dialect
  - ✓ Thêm dependency trong file **pom.xml** và sync lại **maven**

```
<dependency>
<groupId>nz.net.ultraq.thymeleaf</groupId>
  <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

  - ✓ Cấu trúc thư mục Layout page + Content page

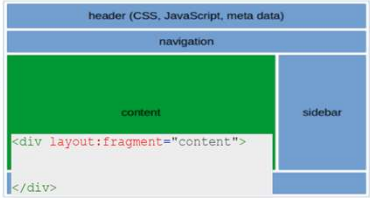
```
templates/
├── layout.html
└── home.html
```

  - ✓ Để sử dụng từ những template khác (như các footer, các header, các menu…).
    - ➢ Thymeleaf cần chúng ta định ra các phần đó, được gọi là các "fragment" -> sử dụng thuộc tính layout:fragment.

SỬ DỤNG THYMELEAF

- Layout page
  - ✓ Thêm thuộc tính xmlns:layout

```
<html
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
```



  - ✓ Sử dụng div với layout:fragment: Chỉ định phần của trang content cho phần nội dung.

SỬ DỤNG THYMELEAF

- Content page
  - ✓ Thêm thuộc tính xmlns:layout và layout:decorate

```
<html lang="en"
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
layout:decorate="_layout">
```

Trong đó: _layout: là tên **Layout page**

  - ✓ Ngoài ra có 2 cách để include content từ fragment :
    - ***insert*** – chèn 1 fragment đã có vào tag hiện tại.
    - ***replace*** – thay thế cho thẻ hiện tại.
  - ✓ Tham khảo: https://www.baeldung.com/spring-thymeleaf-fragments

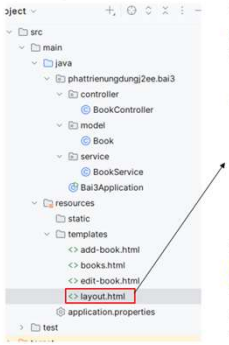
SỬ DỤNG THYMELEAF

- Demo – Quản lý sách sử dụng chung layout (header, footer) cho tất cả các trang

SỬ DỤNG THYMELEAF

❑ Demo – Hướng dẫn

Thiết kế
Layout page

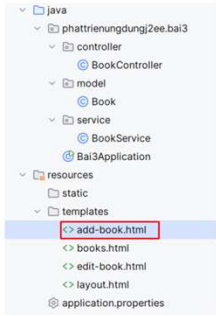



```html
<!DOCTYPE html>
<html xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<head>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" href="/">Home</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav">
                <li class="nav-item">
                    <a class="nav-link" href="/books">Books</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="/about">About</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
<main class="container mt-4">
    <div layout:fragment="content">
    </div>
</main>
<footer class="bg-light text-center py-3 mt-auto">
    <p class="mb-0">© 2025 Phat trien ung dung voi J2EE</p>
</footer>
</body>
</html>
```

SỬ DỤNG THYMELEAF

❑ Demo – Hướng dẫn

Thiết kế
Content page
– "add-book"

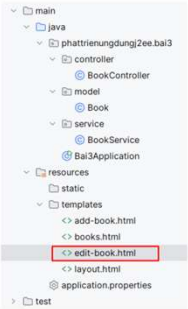



```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layout.html}">
<head>
    <title>Thêm Sách</title>
</head>
<body>
<div layout:fragment="content">
    <h2 class="mb-4">Thêm Sách Mới</h2>
    <form action="/books/add" method="post" th:object="${book}" class="row g-3">
        <!— Tiêu đề sách —>
        <div class="col-md-6">
            <label for="title" class="form-label">Tiêu đề</label>
            <input type="text" id="title" th:field="*{title}" class="form-control"
placeholder="Nhập tiêu đề sách" required>
        </div>
        <!— Tác giả sách —>
        <div class="col-md-6">
            <label for="author" class="form-label">Tác giả</label>
            <input type="text" id="author" th:field="*{author}" class="form-
control" placeholder="Nhập tên tác giả" required>
        </div>
        <!— Nút hành động —>
        <div class="col-12">
            <button type="submit" class="btn btn-success">Thêm</button>
            <a href="/books" class="btn btn-secondary">Hủy</a>
        </div>
    </form>
</div>
</body>
</html>
```

SỬ DỤNG THYMELEAF

❑ Demo – Hướng dẫn

Thiết kế
Content page
– "edit-book"

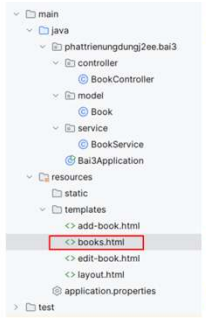



```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layout.html}">
<head>
  <title>Sửa Sách</title>
</head>
<body>
<div layout:fragment="content">
  <h2 class="mb-4">Sửa Thông Tin Sách</h2>
  <form action="/books/edit" method="post" th:object="${book}" class="row g-3">
    <!— ID Sách (ẩn để gửi lên server) —>
    <input type="hidden" th:field="*{id}" />
    <!— Tiêu đề sách —>
    <div class="col-md-6">
      <label for="title" class="form-label">Tiêu đề</label>
      <input type="text" id="title" th:field="*{title}" class="form-control"
placeholder="Nhập tiêu đề sách" required>
    </div>
    <!— Tác giả sách —>
    <div class="col-md-6">
      <label for="author" class="form-label">Tác giả</label>
      <input type="text" id="author" th:field="*{author}" class="form-control"
placeholder="Nhập tên tác giả" required>
    </div>
    <!— Nút hành động —>
    <div class="col-12">
      <button type="submit" class="btn btn-primary">Cập Nhật</button>
      <a href="/books" class="btn btn-secondary">Hủy</a>
    </div>
  </form>
</div>
</body>
</html>
```

SỬ DỤNG THYMELEAF

❑ Demo – Hướng dẫn

Thiết kế
Content page
– "books"




```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layout.html}">
<head>
    <title>Danh Sách Sách</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div layout:fragment="content">
    <h2 class="mb-4">Danh Sách Sách</h2>
    <div class="mb-3">
        <a href="/books/add" class="btn btn-primary">Thêm Sách Mới</a>
    </div>
    <!— Bảng danh sách sách —>
    <table class="table table-bordered table-striped">
        <thead class="table-light">
        <tr>
            <th>ID</th>
            <th>Tiêu đề</th>
            <th>Tác giả</th>
            <th>Hành động</th>
        </tr>
        </thead>
        <tbody>
        <tr th:each="book : ${books}">
            <td th:text="${book.id}"></td>
            <td th:text="${book.title}"></td>
            <td th:text="${book.author}"></td>
            <td>
                <a th:href="@{/books/edit/{id}(id=${book.id})}" class="btn btn-warning
btn-sm">Sửa</a>
                <a th:href="@{/books/delete/{id}(id=${book.id})}" class="btn btn-danger
btn-sm">Xóa</a>
            </td>
        </tr>
        </tbody>
    </table>
</div>
</body>
</html>
```